

Business Process Adaptation on a Tracked Simulation Model

Andrei Solomon
York University
Toronto, Canada
asolomon@cse.yorku.ca

Marin Litoiu
York University
Toronto, Canada
mlitoiu@yorku.ca

Jay Benayon
IBM
Toronto, Canada
jayb@ca.ibm.com

Alex Lau
IBM
Toronto, Canada
alexlau@ca.ibm.com

Abstract

Business processes need to adapt to changes in the operating conditions and to meet the service-level agreements (SLAs) with a minimum of resources. Changes in operating conditions include hardware and software failures, load variation and variations in user interaction with the system. An integral component to adaptation is the awareness over the behavior of self and environment (or having an estimation of the current situation). Aiming at estimation, this paper investigates the automatic building of a dynamic predictive model of the business process that is used for business process optimization. The model is a simulation model whose parameters are tuned at run time by tracking the system with a particle filter.

1 Introduction

Business processes need to adapt to changes in the operating conditions (hardware and software failures, variation in offered load arriving at a business service) and to meet the service-level agreements with a minimum of resources. An adaptive business process monitors Key Performance Indicators (KPIs) through performance measurements (e.g. response time, throughput, or utilization), determines necessary changes to system management variables, and executes them in order to achieve an optimal performance.

The key to a successful adaptation is a simulation model of the business process that allows

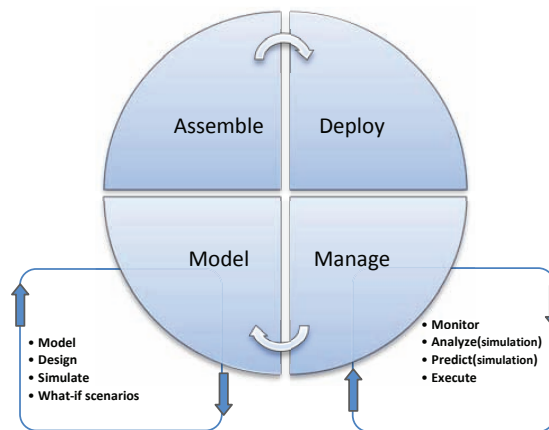


Figure 1: Model-Assemble-Deploy-Manage life cycle of a business process.

for quantitative evaluation of different change decisions. A simulation model can be the basis of an autonomic computing loop on a small time scale or a human assisted life cycle process on a larger time scale. Both are illustrated in Figure 1. A feedback control scheme based on a tracked simulation model can handle the changes in the workload and the system parameters by changing the underlying IT infrastructure, and maintain good KPIs for a business process. Complementarily, a business analyst can use the simulation model to evaluate "what-if" scenarios, and to reassemble and deploy a more optimized version of the process.

In this paper, we show how to accurately tune a simulation model at run time which in turn captures the changes in the workload and the system parameters, and estimates current KPIs for a business process. An estimator tunes the simulation model with the aid of an online monitor so that it accurately matches

Copyright © 2010 Andrei Solomon, Marin Litoiu and IBM Canada Ltd. Permission to copy is hereby granted provided the original copyright notice is reproduced in copies made.

the real process and automatically calibrates the simulation model, minimizing the error between the metrics offered by the two. The estimator eliminates the modeling and measurement errors inherent in any performance analysis.

A simulation model, in sync with the underlying process, and integrated with modeling tools, will speed the adaptation through the process life cycle [18] and enable autonomic computing activities. It automates and speeds up the cycle time between the manage and the model life cycle stages by enabling the delivery of runtime data and statistics into the process modeling environment. At the same time it allows for the completion of the analysis that drives iterative process re-engineering through a continuous business process improvement cycle. "What-if" simulations can be done multiple times using smart search techniques to find the answer to optimization questions [8].

The remainder of the paper is structured as follows. Section 2 presents the background and the related work; Section 3 presents our methodology; Section 4 describes our suggested algorithm; Section 5 presents the results of applying our proposed methodology to a case study; Section 6 describes another approach to our study; Section 7 shows a business process optimization solution; the conclusions and further work are presented in Section 8.

2 Background and Related Work

2.1 Business Process Management

A business process is a series of coordinated tasks or activities, performed by either people or equipment that transform one or more inputs into a defined set of results that are of value to a customer. Business Process Management (BPM) is a discipline whose goal is to systematically organize, manage, analyze, and optimize an organization's business processes. Software tools supporting BPM throughout the business process improvement life cycle, as seen in Figure 1, are called Business Process Management Systems (BPMS).

The BPM life cycle consists of four phases:

- *Model.* In this phase, an "as-is" or "to-be" business process is defined, designed, graphically modeled and simulated in a BPMS. Graphical standards in this stage (such as BPMN or UML AD) allow developers, analysts and business managers to express and understand business processes and their possible flows in a diagrammatic way at a higher level [12].
- *Assemble.* At this stage, the model designed in the previous stage is assembled, developed, configured and tested on the underlying system infrastructure. This stage has no clear standard because of the differences in the IT architectures of different enterprises [12].
- *Deploy.* The *Deploy* phase is where the modeled business process is deployed on an actual BPMS engine and uses the configuration from the *Assemble* phase. BPEL is an example of an execution standard that is commonly used at this stage [12].
- *Manage.* Business processes, after being deployed, are monitored and analyzed with appropriate tools for potential bottlenecks in order to prepare the grounds for the new design phase adaptations in response to the changing environments. The tools to do this are embodied in diagnosis standards [12].

Business Process Modeling Notation (BPMN) gained much acceptance in the industry and many of its implementations can be mapped directly to BPEL code. As a core enabler of BPM, it provides the graphical notation for specifying business processes in Business Process Diagrams (BPD), based on a flowcharting technique very similar to activity diagrams. Business processes can be modeled at different levels of detail, from the perspective of all business stakeholders, which helps reduce the communication gap that frequently occurs between the *Model* and *Assemble* phases. BPMN uses a small set of graphical elements divided in five categories: flow objects (for example events, activities, gateways, also seen in Figure 2), data (e.g.,

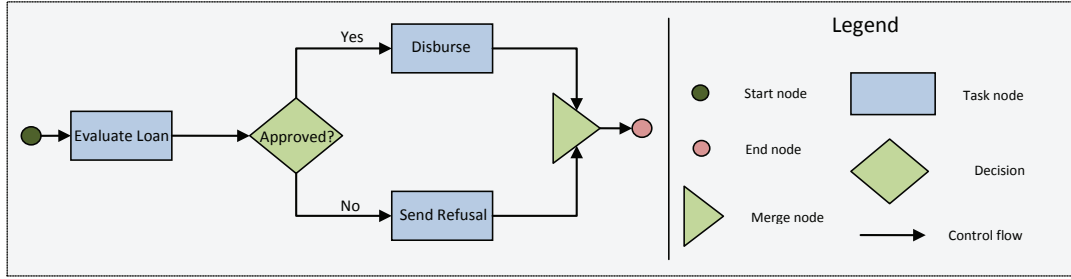


Figure 2: Model phase BPMN diagram of a simple business process.

data objects, data stores), connecting objects (such as sequence flow, message flow, association), swimlanes (e.g., pools, lanes) and artifacts (e.g., group, text annotation).

Business Process Execution Language (BPEL) for Web Services [6], also known as WS-BPEL or BPEL4WS, is an OASIS standard executable language for business processes. It allows composition of web services and is thus the top-down approach to SOA. BPEL follows the orchestration paradigm, where the involved web services do not know (and do not need to know) that they are involved in a composition and that they are part of a higher business process. BPEL is a relatively simple and straightforward way to compose these functionalities in the right order and to modify them, which makes it a more flexible approach compared to choreography. BPEL can express loops and conditional behavior, declare variables, copy and assign values, define fault handlers, and so on. By combining all these constructs, BPEL can define complex business processes in an algorithmic manner. BPEL is not as powerful as JavaTM, but on the other hand, it is simpler and better suited for business process definition.

2.2 Simulation

The lack of tools for evaluating the effects of designed solutions is one of the major problems in business process re-engineering. Mistakes are observed only too late, after implementation, when it is difficult and expensive to correct them [14]. Using a simulation model

to investigate system behavior is less laborious, more flexible, cheaper and safer than experimentation on a real system. The model is an abstraction of a system that avoids unnecessary detail. A simulation model can be simple or complex, but if the state space of the model resembles the real system's state space, often it can answer any questions about the system [17]. Real data is needed in order to build, validate and finally experiment with the model. Today many BPMSs come with monitoring capabilities that make simulation easier and more reliable by avoiding synthetic data generation that mimics the behavior of the process under study. Simulation models can be seen as a "black box", transforming inputs into outputs. Inputs are parameters specified prior to each simulation run and they can describe arrival rates, service times or random number seeds. Outputs, on the other hand, can provide accurate insights about the modeled system and can also help the user focus on the relevant choices that can be made [17]. Simulation models are used because they are fast, they can capture the random behavior of the resources in the system and they can be used as a decision support tool for continuous improvement or adaptation to new competitive conditions [15].

2.3 Particle Filters

Particle filters, also known as sequential Monte Carlo methods (SMC), are model estimation techniques based on simulation with many important applications in robotics, multimedia, military, surveillance and computer vision. Also known as the *survival of the fittest*, the

general idea of the filter comes from the natural way entities evolve. Having a population of sample inputs $\mathbf{x}^{(i)}$ from a known distribution, we can characterize each sample by importance weight $w^{(i)}$, calculated by the observation function $\mathbf{y}^{(i)} = g(\mathbf{x}^{(i)})$. After a number of generations, most successful particles survive by weight recalculation. These weights are used to estimate the final hidden variable $\mathbf{x}$. When the measurement functions are non-linear and the posterior probability of the state is non-Gaussian, conventional filters, such as the Extended Kalman Filter (EKF), may give a large estimation error. Some efforts to improve the EKF have led to the Unscented Kalman Filter (UKF) [11, 16, 10]. The improvement was significant for certain problems, but divergence or poor approximation could still occur in some non-linear problems. Particle filters are a much faster alternative to the Extended Kalman Filter (EKF) or Unscented Kalman Filter (UKF). The accuracy of the algorithm depends on the sample size. With sufficient sample diversity, particle filters can be made more accurate than either of the two Kalman filters. When the simulated sample size is not sufficiently large or lacks diversity among the particles (i.e., contains many repeated points), particle filters might suffer from sample impoverishment [7, 13].

2.4 Automation

Business agility is a success factor for businesses. Many decisions could be made faster through effective use of software tools, appropriate selection of processes and use of prototyping to be able to respond quickly to rapid change in a competitive market [5]. Even though there is an increasing software support for BPM that can help in building a structure of an adaptive business process model, at the same time there is a lack of automation in the BPM life cycle [20].

Enough research into BPM and BPM optimization is available, but there is very limited published material on automation and optimization of BPM. There is no fully automatic method for data acquisition and analysis; however, there are available tools that can help in building the structure of an adaptive busi-

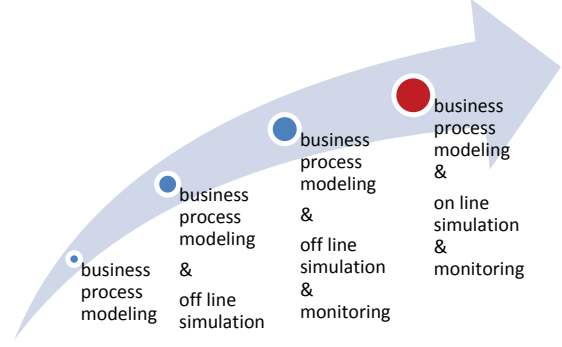


Figure 3: Model and manage phase evolution in the BPM life-cycle.

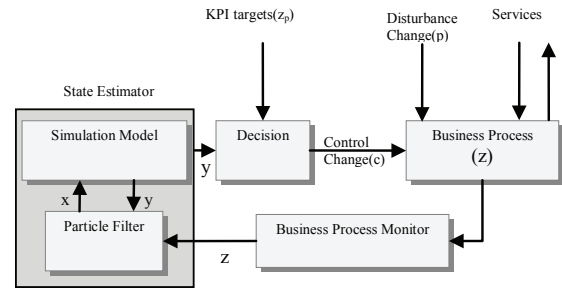


Figure 4: The proposed optimization loop for a business process. A decision Maker changes c , based on a simulation model.

ness process model. Figure 3 depicts the evolution of the *Model* and *Manage* phases of the BPM life cycle. Offline simulation and monitoring features have been gradually added to improve the optimization process of business processes. We consider that automating the transition from the manage to the model phase with online simulation and monitoring would be the next key feature in any BPMS.

3 A Business Optimization Architecture

Self-optimization is the automatic monitoring and control of resources to ensure the optimal functioning with respect to the defined requirements. We propose an optimization architecture based on a feedback loop shown in Figure 4.

A deployed business process makes use of

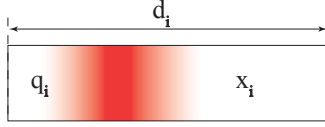


Figure 5: d_i composition. We can measure only d_i , but we do not know how much q_i and x_i are.

service components that can experience external disturbances p . The process and its sub-components are monitored by the Business Process Monitor and recorded for future analysis. It is also meant to provide real-time access to critical business performance indicators (KPIs). In a classical feedback control loop, the monitor is usually referred to as the sensor. System's goal is to maintain monitored properties of the output of the process, z , close to the reference input z_p (the desired KPI targets). The value of z , which might vary because of disturbances, is monitored and fed back usually through some filters (in our case, the particle filter and simulation model) where their output y (estimated performance) is verified against the reference input z_p . The value of x or the simulator input is computed iteratively inside the state estimator until the output of the simulator y matches the monitored z . The value of y (estimated performance), together with a particular control algorithm in the decision block, uses these results to decide on a suitable correction u to bring z closer to z_p using context-specific actuators [9].

4 State Estimator

The state estimator is the key element in the control loop because its goal is to calibrate the state parameters of the model to reflect real system's unpredictable performance drifts. It is composed of a particle filter and a simulator, both working hand in hand to find simulator parameter estimates that cannot be measured. Raw monitored data contains noise and hides other relevant information, such as branching probabilities or task service times without queuing delays. By smoothing it, we obtain more refined data, also called aggregated measurements, that becomes valuable input for our

state estimator when tracking hidden task service times parameters. Together with this new preprocessed data, the state estimator finds the best input for the measured output by correcting the measurements through estimation.

4.1 Monitoring - Monitor Measurements

The monitor records the business process performance via instance metrics that are a collection of measurements, resulting from each execution of the process. An instance metric contains three types of information: task durations, the end-to-end process duration and decision nodes branching. We call the measurement vector $\mathbf{z}$, the vector that contains the task durations vector $\mathbf{d}$ and the end-to-end duration of the execution e : $\mathbf{z} = \langle \mathbf{d}, e \rangle$, where $\mathbf{d} = \langle d_1, \dots, d_n \rangle$. Each $d_i, 1 \leq i \leq n$, represents i^{th} task duration and n is the total number of tasks in the process. All d_i s have two components: the queuing time q_i for the task-specific resources and the actual service time x_i , but both q_i and x_i are unknown to us (Figure 5). Therefore, we measure all $d_i, 1 \leq i \leq n$, and the end-to-end duration e and we want to estimate all $x_i, 1 \leq i \leq n$, at any moment in time to keep the model in sync with the system. Apart from $\mathbf{z}$, an instance metric also contains the chosen path of execution for each decision node during the execution of the process. For instance, for a decision node with a set of branches $\mathcal{B}$, an execution is represented as a vector $\mathbf{b} = \langle b_1, \dots, b_{|\mathcal{B}|} \rangle$ where $b_i = 1$, is the executed branch, and all other $b_k = 0, \forall k \neq i$ represent the un-executed branches at that particular moment in time. To summarize, an instance metric is made up of $\mathbf{z}$ and $\mathbf{b}$.

4.2 Preprocessing - Aggregated Measurements

Aggregate metrics are calculated across multiple instances using a function of interest (e.g., average, maximum, minimum, sum or count) to get other useful information about the process. For instance, branching probabilities at decision nodes are unknown to us, but they can be deduced by creating an observation window that contains the last chosen number of execu-

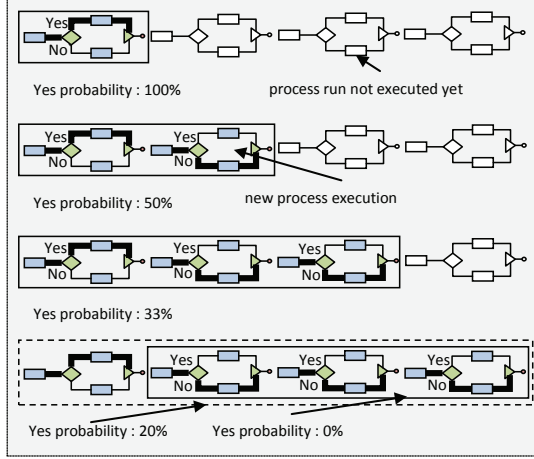


Figure 6: Branch probability is recalculated every time the window slides. The execution path is highlighted in each process instance.

tions as a function of time (e.g., last month) or a predefined number of instances (e.g., last 100 instances). In Figure 6 we show a simple branch probability calculation based on a fixed window of size 3. Note that using a window of size 4 would change the "yes" branch probability of the last execution from 0% to 20%. Also, the window has a smaller size for the first two executions, since there is not enough historical data. The window size affects the aggregated measure. In general, the more samples we have, the more accurate the aggregated metrics are. However, as far as real-world applications are concerned, the algorithm has to work at an adequate speed, trading and balancing accuracy for efficiency. Therefore, the size of the observation window or the time frame of an aggregated measure becomes a tuning parameter.

More formally, the probability calculation process, for one decision node with a $\mathcal{B}$ set of branches, starts with a window of instance metrics. Assume we are at time step t and we want to include in our window the last W branch components $\mathbf{b}$ of the instances for a specific decision node: $\langle \mathbf{b}_{t-W-1}, \dots, \mathbf{b}_t \rangle$.

$$\begin{aligned} \mathbf{b}_{t-W-1} &= \langle b_1 \quad \dots \quad b_k \quad \dots \quad b_{|\mathcal{B}|} \rangle \\ \dots &= \begin{matrix} \vdots & \ddots & \vdots & \ddots & \vdots \end{matrix} \\ \mathbf{b}_t &= \langle b_1 \quad \dots \quad b_k \quad \dots \quad b_{|\mathcal{B}|} \rangle \end{aligned}$$

Each $\mathbf{b}_i \in \langle \mathbf{b}_{t-W-1}, \dots, \mathbf{b}_t \rangle, t-W-1 \leq i \leq t$ is a vector of the form $\langle b_1, \dots, b_{|\mathcal{B}|} \rangle$ for the decision node with a $\mathcal{B}$ set of branches. We refer to a branch $b_k \in \langle b_1, \dots, b_{|\mathcal{B}|} \rangle$ at time step $i, t-W-1 \leq i \leq t$ as $b_{i,k}$. Therefore vector $\langle \mathbf{b}_{t-W-1}, \dots, \mathbf{b}_t \rangle$ is represented in the following matrix:

$$\begin{bmatrix} b_{t-W-1,1} & \dots & b_{t-W-1,k} & \dots & b_{t-W-1,|\mathcal{B}|} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{i,1} & \dots & b_{i,k} & \dots & b_{i,|\mathcal{B}|} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ b_{t,1} & \dots & b_{t,k} & \dots & b_{t,|\mathcal{B}|} \end{bmatrix}$$

Thus, for branch k at time step t and a window of size W , the probability that branch k is selected for of execution is:

$$P(\text{branch} = k) = \frac{\sum_{i=t-W-1}^t b_{i,k}}{W} \quad (1)$$

With equation 1 we can calculate all probabilities $\mathbf{P}_b$ for any given decision node $b \in D$ with a $\mathcal{B}$ set of branches, where D is the set of decision nodes of the process. Note that, $\mathcal{B}$ is specific to each decision node b . All branch probabilities are put into vector $\mathbf{P}_b = \langle P(\text{branch} = 1), \dots, P(\text{branch} = |\mathcal{B}|) \rangle$ and finally all $\mathbf{P}_b, b \in D$ are placed in a vector $\mathbf{P} = \langle P_1, \dots, P_b, \dots, P_{|D|} \rangle$.

Similarly, we take the $\mathbf{z}$ components of the last W instances at time step t , $\langle \mathbf{z}_{t-W-1}, \dots, \mathbf{z}_t \rangle$, and we denote each $\mathbf{d}$ and e subcomponents of $\mathbf{z}_i$ as $\mathbf{d}_i$ and e_i , respectively, $\forall i, t-W-1 \leq i \leq t$:

$$\begin{aligned} \mathbf{z}_{t-W-1} &= \langle \mathbf{d}_{t-W-1}, e_{t-W-1} \rangle \\ \vdots &= \vdots \\ \mathbf{z}_i &= \langle \mathbf{d}_i, e_i \rangle \\ \vdots &= \vdots \\ \mathbf{z}_t &= \langle \mathbf{d}_t, e_t \rangle \end{aligned} \quad (2)$$

Each $\mathbf{d}_i \in \mathbf{z}_i$ is of the form $\langle d_1, \dots, d_n \rangle$, but we want to name task node $d_k \in \langle d_1, \dots, d_n \rangle, 1 \leq k \leq n$, at time step $i, t-W-1 \leq i \leq t$ as $d_{i,k}$ so we can formulate $\overline{d_k}$ as in equation 4 using:

$$\begin{aligned}
& \left[\begin{array}{ccc} d_{t-W-1,1} & \cdots & d_{t-W-1,n} \\ \vdots & \ddots & \vdots \\ d_{i,1} & \cdots & d_{i,n} \\ \vdots & \ddots & \vdots \\ d_{t,1} & \cdots & d_{t,n} \end{array} \right] , \quad \left[\begin{array}{c} e_{t-W-1} \\ \vdots \\ e_i \\ \vdots \\ e_t \end{array} \right] \\
& \langle \bar{d}_1 \quad \cdots \quad \bar{d}_n \rangle , \quad \langle \bar{e} \rangle \\
& \langle \sigma(d_1) \quad \cdots \quad \sigma(d_n) \rangle
\end{aligned} \tag{3}$$

$$\bar{d}_k = \frac{\sum_{i=t-W-1}^t d_{i,k}}{W} \quad \bar{e} = \frac{\sum_{i=t-W-1}^t e_i}{W} \tag{4}$$

In a similar manner, $\bar{d}_k$ and $\sigma(d_k)$ represent the mean and standard deviation for task node k duration over the last W number of instances, respectively. We formulate $\bar{e}$ as the average end-to-end duration of the process in equation 5. In conclusion, at the end of this stage we have aggregated $\mathbf{z}$ across multiple instances using an average function and we ended up with $\bar{\mathbf{z}} = \langle \bar{\mathbf{d}}, \bar{e} \rangle$, where $\bar{\mathbf{d}} = \langle \bar{d}_1, \dots, \bar{d}_n \rangle$. Also, it is important to mention an equally relevant vector, $\mathbf{P}$, derived from multiple $\mathbf{b}$ vectors of the instance metrics.

4.3 Inputs

State estimator inputs are divided between its two components: the simulator and the particle filter. The simulator requires an inter-arrival time A , the number of executions that are separated by A in time, the average task service time $\bar{\mathbf{x}}$ and the probabilities for the decision nodes $\mathbf{P}$ specifications. Because we work with aggregated measures derived from a window of interest, we denote W , the size of the window, to represent the number of executions and A to be computed as the average distance in time between window instances. We define $\bar{\mathbf{x}}$ to be the vector containing only the service time components $\bar{x}_j$ of each $\bar{d}_j \in \bar{\mathbf{d}}$, $1 \leq j \leq n$, as described in equation 6.

$$\begin{aligned}
\bar{\mathbf{d}} &= \langle \bar{d}_1, \dots, \bar{d}_j, \dots, \bar{d}_n \rangle \\
&= \langle \bar{q}_1 + \bar{x}_1, \dots, \bar{q}_j + \bar{x}_j, \dots, \bar{q}_n + \bar{x}_n \rangle \\
&= \langle \bar{q}_1, \dots, \bar{q}_j, \dots, \bar{q}_n \rangle + \langle \bar{x}_1, \dots, \bar{x}_j, \dots, \bar{x}_n \rangle \\
&= \bar{\mathbf{q}} + \bar{\mathbf{x}}
\end{aligned} \tag{6}$$

On the other hand, the particle filter input requires only that the standard deviation of monitored task durations $\langle \sigma(d_1), \dots, \sigma(d_n) \rangle$ to be readily available in a vector $\sigma(\mathbf{d})$ for particle noise generation in the prediction stage (subsection 4.6) of the proposed algorithm.

4.4 Particle Filter

We develop a "model-tuning" algorithm whose goal is to keep the simulation model accurate. The algorithm compensates for the measurement (we know $\bar{d}_j$, but we need its $\bar{x}_j$ component for accurate simulations) and modeling errors (probabilities and inter-arrival times, due to window size) mentioned above, and gets the best estimate for data such that a square root error (for example) between the simulated and measured metrics is minimized. The model tuning works as follows: first we monitor observable system parameters ($\bar{\mathbf{z}}$) to obtain the states of the system; then we track the system's hidden parameters (also referred to as state parameters in state-space model $\bar{x}_j$) with the aid of the monitored data from step one and an estimator (such as a tracking filter). The state and observation equations are provided in equations (7) and (8), respectively:

$$x_k = f(x_{k-1}, u_k) \tag{7}$$

$$y_k = g(x_k, v_k) \tag{8}$$

The goal of particle filtering is to estimate the state variables $\bar{\mathbf{x}} = \langle \bar{x}_1, \dots, \bar{x}_n \rangle$ from a set of observations $\bar{\mathbf{z}}$ that come sequentially over time. Multiple copies of $\bar{\mathbf{x}}$ are used, $\bar{\mathbf{x}}^{(i)}$, $1 \leq i \leq m$, each associated with a weight that signifies the importance of that specific particle, $w^{(i)}$. An overall estimate of the state of the system is obtained by the weighted sum of all the particles. The particle filter algorithm is recursive in nature and operates in two phases: *prediction* and *update*. In order to simulate the

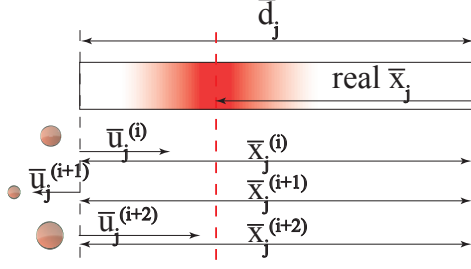


Figure 7: Estimating the real $\bar{x}_j$ for task j from a set of particles.

effect of noise on $\bar{x}$, each particle is updated with the estimated system state variable (prediction stage) and some random noise. Each particle is simulated, the result is compared to the last observation and its weight is reevaluated (update stage). A new estimation of the system state $\bar{x}$ is obtained by the weighted sum of the new particle weights and a new generation of particles is updated using this last estimation.

4.5 Algorithm initialization

Initialization takes place within lines 1 - 5 of Algorithm 1. Consider an execution at time t when we want to find the state variables $\bar{x} = \langle \bar{x}_1, \dots, \bar{x}_n \rangle$ from $\bar{d} = \langle \bar{d}_1, \dots, \bar{d}_n \rangle$ calculated using the last W window. We need to initialize the particles with suitable values for each $\bar{x}_j \in \bar{x}$, $1 \leq j \leq n$, so that the algorithm converges in a finite number of steps K to a value within ET threshold away from the measurement $\bar{e} \in \bar{z}$.

The right starting point would be to initialize all state vector $\bar{x} = \langle \bar{x}_1, \dots, \bar{x}_n \rangle$ components with the aggregated task durations provided by input vector $\bar{d} = \langle \bar{d}_1, \dots, \bar{d}_n \rangle$. Therefore, each $\bar{x}_j \in \bar{x}$ is set to $\bar{d}_j \in \bar{d}$, $1 \leq j \leq n$ (line 4 of Algorithm 1), the overall mean duration for task $j \in N$ including the queuing delay $\bar{q}_j$.

4.6 Algorithm Prediction

Lines 7 - 12 capture the prediction stage of Algorithm 1. Having the state vector $\bar{x} = \langle \bar{x}_1, \dots, \bar{x}_n \rangle$ initialized, we need to add the noise $u_j^{(i)}$ to each estimation variable $\bar{x}_j \in \bar{x}$ according to its own standard deviation $\sigma(d_j)$,

Algorithm 1 Particle Filter Optimization

Input:

vector of average task durations	$\bar{z}$
vector of decision nodes probabilities	$\mathbf{P}$
number of tokens	W
inter-arrival time	A
vector of standard deviations	$\sigma(\mathbf{d})$
error threshold	ET

Output:

vector of average task durations	$\mathbf{y}$
----------------------------------	--------------

```

1: //Initialization:
2: k = 0
3: for all tasks  $j \in N$  do
4:    $\bar{x}_j = \bar{d}_j, \bar{d}_j \in \bar{d}, \bar{d} \in \bar{z}, \bar{x}_j \in \bar{x}$  //initialize
   state
5: end for
6: repeat
7:   for all particles  $i \in M$  do
8:     for all tasks  $j \in N$  do
9:        $u_j^{(i)} = N(0, \sigma(d_j)^2), u_j^{(i)} \in \mathbf{u}^{(i)}$ 
       //Generate noise
10:       $\bar{x}_j^{(i)} = \bar{x}_j + u_j^{(i)}, \bar{x}_j^{(i)} \in \bar{x}^{(i)}, \bar{x}_j \in \bar{x}$ 
       //Generate particle
11:    end for
12:  end for
13:  for all particles  $i \in M$  do
14:     $y^{(i)} = Sim(\bar{x}^{(i)}, A, W, \mathbf{P})$  //Predict
15:     $\varepsilon^{(i)} = |y^{(i)} - \bar{z}|$  //Calculate Error
16:     $w^{(i)} = \frac{1}{\varepsilon^{(i)^2}}$  //Update weights
17:     $w^{(i)} = \frac{w^{(i)}}{\sum_{(\forall i \in M)} w^{(i)}}$  //Normalize
    weights
18:  end for
19:  for all tasks  $j \in N$  do
20:     $\bar{x}_j = \sum_{(\forall i \in M)} w^{(i)} \bar{x}_j^{(i)}$  //Recalculate
     $\bar{x}_j$ 
21:  end for
22:   $\bar{x} = \langle \bar{x}_1, \dots, \bar{x}_n \rangle$  //Update  $\mathbf{x}$ 
23:  for all particles  $i \in M$  do
24:     $\bar{x}^{(i)} = \bar{x}$  //Update particle  $\bar{x}^{(i)}$ 
25:  end for
26:  k = k + 1 //Increment iteration
27: until  $(|Sim(\bar{x}, A, W, \mathbf{P}) - \bar{z}| \leq ET)$  or  $(k > K)$ 
28: return  $\mathbf{y} = Sim(\bar{x}, A, W, \mathbf{P})$ 

```

$1 \leq j \leq n$, in order to create diversity. We generate a set of m white noise (normal distribu-

tion) vectors $\mathbf{u}^{(i)} = \langle u_1^{(i)}, \dots, u_n^{(i)} \rangle$ where each subcomponent $u_j^{(i)} \in \mathbf{u}^{(i)}, 1 \leq j \leq n$, has a 0 mean and $\sigma(d_j)^2$ variance (line 9). We apply the noise to each task $\bar{x}_j^{(i)} = \bar{x}_j + u_j^{(i)}$ to obtain the particle vector $\bar{\mathbf{x}}^{(i)} = \langle \bar{x}_1^{(i)}, \dots, \bar{x}_n^{(i)} \rangle$ (line 10.). On algorithm line 14, each particle vector $\bar{\mathbf{x}}^{(i)}$ is used as input in the simulator. The simulator also needs the inter-arrival time A , the number of requests W and decision nodes probabilities vector $\mathbf{P}$ to be specified. An example regarding the algorithm prediction stage is shown in Figure 7, where the real $\bar{x}_j$ duration for task j is estimated from a set of particle subcomponents $\{\bar{x}_j^{(i)}, \bar{x}_j^{(i+1)}, \bar{x}_j^{(i+2)}\}$. As described in the previous section, all subcomponents for task j are all initialized with $\bar{d}_j$. Particle subcomponent with noise $\bar{x}_j^{(i+2)} + u_j^{(i+2)}$ is the closest to the real $\bar{x}_j$. Using it as input in the simulator would produce better results than using the other two. Therefore, during the update stage, this particle would produce the smallest error and be assigned the highest weight.

4.7 Algorithm Update

Starting with line 15, Algorithm 1 enters the update stage, which lasts until line 25. All m simulation results $\langle y^{(1)}, \dots, y^{(m)} \rangle$ are compared against $\bar{\mathbf{z}}$ in order to compute the errors $\varepsilon = \langle \varepsilon^{(1)}, \dots, \varepsilon^{(m)} \rangle$ (line 15). Simulation results $y^{(i)}, 1 \leq i \leq m$, and the measurement $\bar{\mathbf{z}}$ are vectors that contain an end-to-end duration component $\bar{e}^{(i)}$ and $\bar{e}$, respectively. For each particle $i, 1 \leq i \leq m$, the error $\varepsilon^{(i)}$ becomes the result of $|\bar{e}^{(i)} - \bar{e}|$. Also, the weight vector $w = \langle w^{(1)}, \dots, w^{(m)} \rangle$ has to be reevaluated since the particles $x^{(i)}$ and their simulation outputs $y^{(i)}$ are not the same anymore, producing different errors $\varepsilon^{(i)}$. We chose to emphasize the importance of good particles by using $w^{(i)} = \frac{1}{\varepsilon^{(i)2}}$ instead of $\frac{1}{\varepsilon^{(i)}}$, which gave slower convergence and therefore sometimes poorer results for the same number of iterations. After the weight normalization, we update the estimation components $\bar{x}_j$ in $\bar{\mathbf{x}}, 1 \leq j \leq n$ (line 20.), by the weighted sum of the new contributions $w^{(i)}$ of each particle and their old values $\bar{x}_j^{(i)}$.

Table 1: Notation

W	the set of process instances in the sliding window
N	the set of task nodes and links
D	the set of decision nodes
M	the set of particles
A	mean inter-arrival time for a window size $ W $
$\mathbf{x}$	state vector
$\mathbf{z}$	observation vector
$\mathbf{x}^{(i)}$	i^{th} particle
$w^{(i)}$	weighting coefficient of i^{th} particle
$f()$	system transition function
$g()$	observation function
$\bar{e}$	mean end-to-end process duration
n	total number of tasks in N , the equivalent of $ N $
d_j	j^{th} task duration, $1 \leq j \leq n$
x_j	j^{th} task estimated duration, $1 \leq j \leq n$
$\mathbf{P}_i$	i^{th} decision node vector of branching probabilities
$\mathbf{P}$	the vector containing all $\mathbf{P}_i, 1 \leq i \leq b$
$\bar{d}_j$	mean duration for d_j over a window of size $ W $
$\sigma(d_j)$	standard deviation for d_j over a window of size $ W $

4.8 Algorithm Output

After the loop exit condition is satisfied, at line 27 in Algorithm 1, $\mathbf{y}$ is returned. $\mathbf{y} = \langle \bar{\mathbf{x}}, \bar{e} \rangle$, where $\bar{\mathbf{x}}$ is the task durations vector extracted by the state estimator from $\bar{\mathbf{d}} \in \bar{\mathbf{z}}$ by filtering out the queuing delays $\bar{\mathbf{q}}$. The second component $\bar{e}$ is the end-to-end simulated process duration vector using $\bar{\mathbf{x}}$ as input. These states, $\bar{\mathbf{x}}$, are useful to the decision block of the feedback loop because they can help automatically adapt the process for possible heavy loads.

5 Case Study

In this section we apply our methodology to a real business process and investigate how much the algorithm improves the simulation model, through estimation. First we describe the tools used during experimentation and then the selected scenarios for real-time monitoring and estimation.

5.1 The Setup

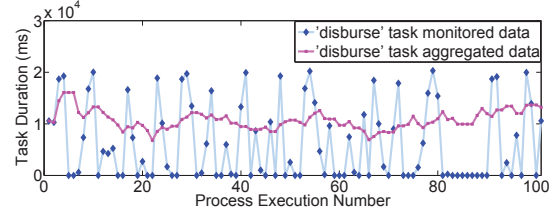
We used IBM® WebSphere Business Modeler Advanced® [1] to design, simulate and analyze a business process under a wide variety of conditions. Modeler can also transform the process into BPEL for implementation, generate reports and redefine the existing and pro-

posed processes together with the required resources and business items. Well-defined processes can help locate bottlenecks through simulation and remove them in the model stage of the BPM life cycle. For monitoring, we used WebSphere Business Monitor[®] [2] on a WebSphere Process Server[®] [4]. It is an application that is deployed on a server to collect business performance data during the BPM manage life-cycle phase and to provide it for analysis via dashboards or REST APIs to the user. Built on top of WebSphere Application Server[®], WebSphere Process Server[®] supports traditional business integration and advanced business process solutions based on service-oriented architecture (SOA). Finally, WebSphere Integration Developer[®] [3] was used for assembling (assemble BPM life-cycle phase) and deployment (deploy BPM life-cycle phase) of business processes. WebSphere Integration Developer[®] is a tool for business process integration, allowing for flexible service components wiring and rapid composite application development [19].

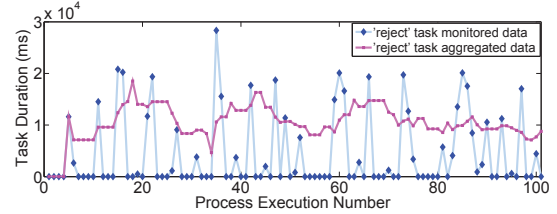
5.2 Process Workload Generation

With the aid of IBM WebSphere Modeler[®], we designed a simple process similar to the one in Figure 2, containing only three tasks and a decision node. In order to observe how the state estimator responds to load variation, each task is automated by a web service and their service time is controlled to follow a sine function, as seen in the monitored durations of Figure 8(c). The other two monitored web services, shown in Figure 8(a) and 8(b), follow the same pattern. In reality these service demands do not have the high variation we generated in these experiments. However, we consider them here to stress the algorithm.

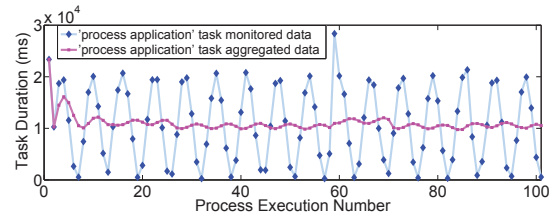
The aggregated values are calculated as described in section 4.2 together with a window of size 12. The particle filter was introduced into the feedback loop to estimate under any conditions, including variable window sizes. The algorithm works with any window size as long as it captures enough diversity for meaningful standard deviations. Usually, the larger the window, the more accurate the probabilities and estimations are, but this is beyond the goal



(a) Monitored and aggregated data for web-service 'Disburse'.



(b) Monitored and aggregated data for web-service 'Reject'.



(c) Monitored and aggregated data for web-service 'Process Application'.

Figure 8: Generated workload is monitored and preprocessed before feeding into the State Estimator.

of this paper.

5.3 Results

Workloads were designed to explore different scenario conditions by capturing arrival rate (or inter-arrival time) variation with uniform service time for each task, service time variation with uniform arrival rate and both inter-arrival time and service time variation.

The size of sliding window can be a function of time (e.g., last week) or simply a predefined number (e.g., last 100 instances). In this paper, the focus is on short-term trends, therefore several hundred workload instances have been generated to test the state estimator performance.

Our experiment is intended for real-time monitoring and therefore starts with no histor-

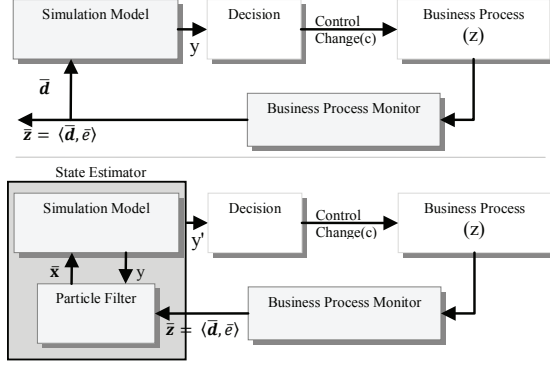


Figure 9: y' and y represent output of the simulation model with and without a particle filter.

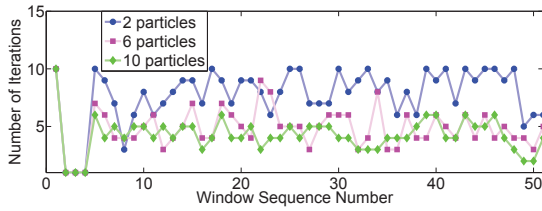


Figure 10: Number of iterations needed to achieve an accurate estimation.

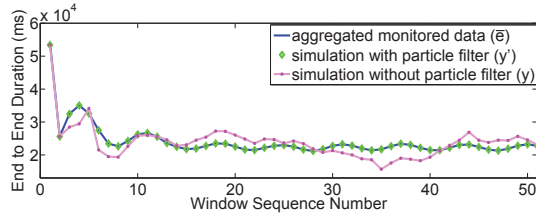


Figure 11: Offline State Estimation.

ical data. The window grows until the size of historical data builds up and matches the maximum size of the window W (i.e., Figure 6), in our case 12. After that, the window slides with every process instance introduced into the Monitor.

We consider two cases, illustrated in Figure 9, where the simulator outputs y and y' are used by the decision block algorithm for business process adaptation to environmental change. The vector y is derived from a simple insertion of the $\bar{d}$ vector into the simulator (without a particle filter). The vector $\bar{d}$ contains both $\bar{q}$, the queuing time, and $\bar{x}$, the ser-

vice time vector. On the other hand y' is the product of the state estimator (simulator and particle filter) after finding $\bar{x}$ in a finite number of iterations between two consecutive process executions. Then, we compare the results (Figure 11) of simulation with and without a particle filter.

We define the modeling error as the absolute difference between the end-to-end simulated (y) and measured response times ($\bar{e}$) averaged over the number of window sequences. For the experiments where no particle filter is used or where the $\bar{d}$ is employed instead of service times $\bar{x}$, the modeling error is 1411 ms. On the other hand, for the experiments where the particle filter is used (y'), the situation is a lot better. At time $t - 1$ the system is in a state where the aggregated measures are $\bar{z}_{t-1} = \langle \bar{d}_{t-1}, \bar{e}_{t-1} \rangle$ and $\mathbf{P}_{t-1}$. At time step t it receives a new request, which is inserted into the monitor. The sliding window acts like a fixed size FIFO queue and adds this new process instance while removing the oldest one. Preprocessing follows and a new $\bar{z}_t = \langle \bar{d}_t, \bar{e}_t \rangle$ and $\mathbf{P}_t$ is derived and fed to the state estimator Algorithm 1. The algorithm runs for a maximum of $K = 10$ iterations and returns y' . Figure 10 shows the number of iterations required to reach an optimal solution $\bar{x}$ to our estimation problem. On average the algorithm iterates 7.72 times with 2 particles, 4.84 times with 6 particles and finally 4.25 times with 10 particles. The average distance between y' and the aggregated measured end-to-end duration is 0.96 ms for 2, 0.48 ms for 6 and 0.44 ms for 10 particles. When a new process instance is executed at time step $t + 1$, the window shifts to include it, $\bar{z}_{t+1}$ and $\mathbf{P}_{t+1}$ are evaluated and the state estimator iterates again through the algorithm.

6 Fast changes in parameters

In the previous section we described an algorithm that executes every time a new process instance metric arrives in the monitor. The assumption was that there is enough time for estimation between any two measurements. In situations with a high volume of process exe-

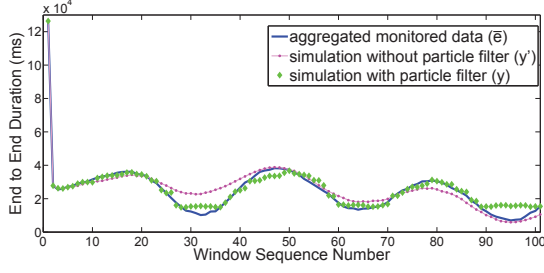
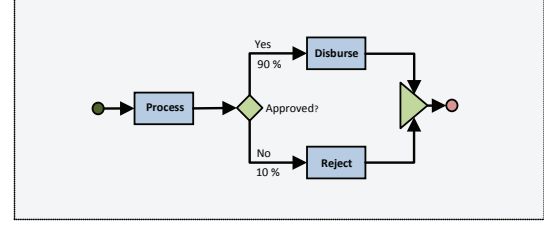


Figure 12: Online State Estimation.

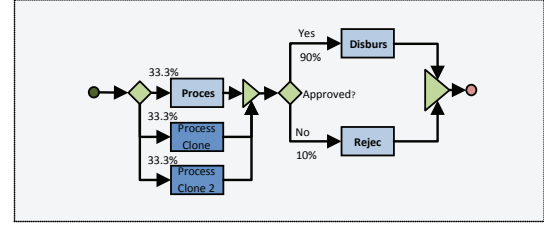
cutions, we need a faster approach. Therefore, we use Algorithm 1 with only one iteration instead. We initialize the algorithm with the first aggregated measurement $\bar{z}$ for the first algorithm execution. Subsequent executions at any time step $t > 1$ use the estimations $\bar{x}_t^{(i)}$ at time step $t+1$ for each particle initialization. In Figure 12 the simulation with particle filter y' outperforms the regular simulation without particle filter aid. The average modeling error of the online state estimator is 1865 ms while simulation without it cannot do better than 3420 ms on average. We used the similar stress workloads described in subsection 5.2. They show the trade-off between accuracy and speed: the higher the variation in arrival rate or service time, the less accurate the online estimation and simulation model become.

7 Using simulation to improve the process

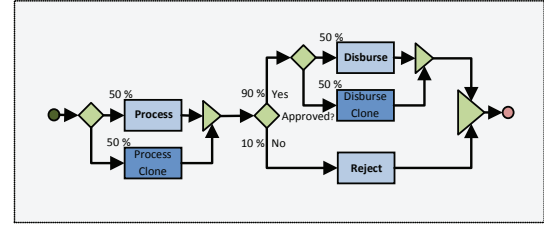
Business process adaptation can be achieved from any configuration by adding more resources to the initial one. At run time, the feedback loop decision block can use the simulator on a currently deployed business process to execute possible future scenarios in order to find the bottlenecks that depreciate the performance of the system. For instance, a simple business process can be deployed entirely on a single physical machine, including its web services. Poorly performing tasks can be found with the aid of the simulator by running different possible inter-arrival times for the arriving requests at the process server. All web services can be sorted dynamically by queuing delay in descending order using the information fetched



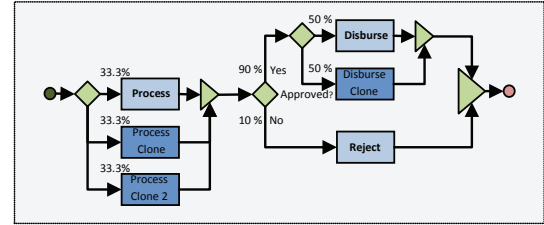
(a) Original configuration.



(b) 2 clones are added to worst performing task.



(c) Distributing clones between worst performers



(d) Configuration (b) and (c) combined.

Figure 13: Simulated system configurations.

from simulation reports. Possible resource allocation configurations are tried by selecting the worst-performing task and by adding clones to the process. Their web service implementation can be dynamically deployed on other machines to take over part of the workload and maintain the service-level agreement (SLA) or KPIs within acceptable limits. As an example, in Figure 13 (a) we present a business process with three tasks, each one being automated by

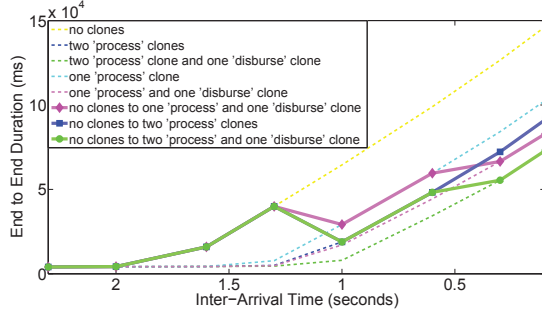


Figure 14: Scaling with different configurations.

a web service: "Process Loan", "Disburse" and "Reject", respectively. Assuming that the first two activities have the highest queuing delay, they are cloned and inserted to work in parallel with the original ones in Figure 13 (c). Another option is checking the configuration with two extra copies of the worst-performing task in Figure 13 (b). Finally, we also consider combining (c) and (b) in Figure 13 (d). These four configurations are represented in Figure 14 by the dotted lines together with an extra transition configuration with one clone for the "process" task. Simulation results show no queuing delay for configuration (a) at inter-arrival times greater than 2 seconds and at 1.3 second for (d). At inter-arrival time of 1 second, we create the transition configuration by adding a copy of the worst performer from configuration (a) to decrease the response time of the process. By adding two clones instead, we do not improve the end-to-end duration by much. In Figure 14 we observe that it is actually better to add only one copy of the first- and second-worst performers, by comparing dotted line "one 'process' and one 'disburse' clone" with the "two 'process' clones" one. Depending on the resource costs and KPIs, we can select the most convenient configurations by computing the gain-to-resource cost ratio.

8 Conclusions and future work

We took a step towards correct estimation of performance parameters for business process optimization in BPM. We applied our track-

ing technique on a simulated system with random service times and sinusoidal loads. It was shown that despite parameter changes, the particle filter could track them successfully. We also investigated a possible business process optimization approach to keep the business process response time under the SLA threshold with minimum resources and cost. The implementation was done through simulation of monitored business processes and we used IBM® WebSphere Modeler Advanced®, WebSphere Modeler Simulator®, WebSphere Integration Developer® and WebSphere Business Monitor® on a WebSphere Process Server®.

For future work we would like to apply regression techniques that could be used to obtain more accuracy in trend prediction. For example, we would use Linear Regression and Autoregressive Models to exploit the trade-off between a simple description of overall data trends and the accuracy of predictions made from the model.

We would also extend the KPI trend analysis to include cyclical patterns (e.g., weekly and monthly patterns), the model-tuning algorithm to take into account the data distribution, and implement an optimization algorithm. Evaluation on real applications would be desirable. Developing a trend analysis module that takes into account the historical data and the correlation between metrics (as captured by the simulation model) would identify daily, weekly and monthly trends and use those to forecast KPIs. Implementing the optimization algorithm that will use the simulation model and current and forecasted KPIs is meant to optimize a business goal by tuning or reallocating resources or changing business rules. Furthermore, optimization techniques leveraging simulation could be exploited to determine what to change to minimize or maximize a KPI while the process is being monitored to better guide the business user with options they could consider.

9 Acknowledgments

This research was supported by OCE, the Ontario Centres of Excellence and by the IBM Toronto Centre for Advanced Studies, as part

of the program of the Centre for Research in Adaptive Systems (CERAS).

10 About the Authors

Andrei Solomon is a masters student at York University, Canada. He is an IBM CAS student since 2009 and he worked in the IBM Toronto Lab doing research for his masters degree. His research interests include autonomic computing, distributed systems and performance engineering.

Marin Litoiu is a professor at York University. Prior to that he was a Senior Research Staff Member with Centre for Advanced Studies, IBM Toronto Lab, where he led the research programs in Autonomic Computing, System Management and Software Engineering. He was the Chair of the Board of CSER, a Canadian Consortium for Software Engineering Research and Director of Research for Centre of Excellence for Research in Adaptive Systems. Dr. Litoiu holds doctoral degrees from University Polytechnic of Bucharest and from Carleton University. His research interests include autonomic computing; high performance software design; performance modeling, performance evaluation and capacity planning for distributed and real time systems.

Jay Benayon is a senior manager in the WebSphere Business Modeler organization at the IBM Software Group Toronto Laboratory where he leads the development of the simulation, analytics and optimization components. Jay joined IBM in 1988 and holds a bachelor's and a master's degree in computer science. He is an IBM Master Inventor with over 18 patents (pending) and has published several research papers related to business process simulation and optimization.

Alex Lau is a research staff member at IBM CAS Canada Research, where his research focuses on service oriented architecture, BPM tools, and business events. Prior to joining CAS, Alex was a software engineer in IBM Canada, where he worked on a number of WebSphere BPM products including Websphere Integration Developer and Websphere Business Modeler. Alex holds a Bachelor's degree in Mathematics (Computer Science) and a Mas-

ter's degree in Mathematics (Computer Science) from the University of Waterloo.

References

- [1] <http://www.ibm.com/software/integration/wbimodeler/>. 2009.
- [2] <http://www.ibm.com/software/integration/wbimonitor/>. 2009.
- [3] <http://www.ibm.com/software/integration/wid/>. 2009.
- [4] <http://www.ibm.com/software/integration/wps/>. 2009.
- [5] M. Al-Mashari and M. Zairi. BPR implementation process: an analysis of key success and failure factors. *Business Process Management Journal*, 5(1):87–112, 1999.
- [6] T. Andrews, F. Curbera, H. Dholakia, Y. Golland, J. Klein, F. Leymann, K. Liu, D. Roller, D. Smith, and S. Thatte. Business process execution language for web services specification, version 1.1. *IBM, Microsoft, BEA, SAP and Siebel Systems*, 2003.
- [7] M. S. Arulampalam, S. Maskell, N. Gordon, T. Clapp, D. Sci, T. Organ, and S. A. Adelaide. A tutorial on particle filters for online nonlinear/non-GaussianBayesian tracking. *IEEE Transactions on signal processing*, 50(2):174–188, 2002.
- [8] M. Barnett. Modeling & simulation in business process management. *BP Trends Newsletter, White Papers & Technical Briefs*, 2003.
- [9] Y. Brun, G. Di Marzo Serugendo, C. Gacek, H. Giese, H. Kienle, M. Litoiu, H. Muller, M. Pezze, and M. Shaw. Engineering self-adaptive systems through feedback loops. *Software Engineering for Self-Adaptive Systems*, pages 48–70, 2009.
- [10] S. J. Julier and J. K. Uhlmann. A new extension of the kalman filter to nonlinear systems. In *Int. Symp. Aerospace/Defense Sensing, Simul. and Controls*, volume 3, page 26. Citeseer, 1997.

- [11] S. J. Julier, J. K. Uhlmann, and H. F. Durrant-Whyte. A new approach for filtering nonlinear systems. In *Proceedings of the American Control Conference*, volume 3, page 16281632. American Automatic Control Council, Evanston, IL, 1995.
- [12] R. K. L. Ko, S. S. G. Lee, and E. W. Lee. Business process management (BPM) standards: a survey. *Business Process Management Journal*, 15(5), 2009.
- [13] J. H. Kotecha and P. M. Djuric. Gaussian particle filtering. *IEEE Transactions on Signal Processing*, 51(10):2592–2601, 2003.
- [14] E. Paolucci, F. Bonci, and V. Russi. Redesigning organisations through business process re-engineering and object-orientation. In *Proceedings of the European Conference on Information Systems*, pages 587–601, 1997.
- [15] R. J. Paul, V. Hlupic, and G. Giaglis. Simulation modeling of business processes. In *Proceedings of the 3rd UK Academy of Information Systems Conference*, pages 311–320. Citeseer, 1998.
- [16] A. Romanenko and JAAM Castro. The unscented filter as an alternative to the EKF for nonlinear state estimation: a simulation case study. *Computers & chemical engineering*, 28(3):347–355, 2004.
- [17] P. J. Sanchez. Fundamentals of simulation modeling. In *Proceedings of the 39th conference on Winter simulation: 40 years! The best is yet to come*, pages 54–62. IEEE Press, 2007.
- [18] L. Verner. BPM: the promise and the challenge. *Queue*, 2(1):91, 2004.
- [19] U. Wahli, V. Avula, H. Macleod, M. Saeed, and A. Vinther. Business process management: Modeling through monitoring using WebSphere v6. 0.2 products.
- [20] B. Wetzstein, Z. Ma, A. Filipowska, M. Kaczmarek, S. Bhiri, S. Losada, J. M. Lopez-Cobo, and L. Cicurel. Semantic

business process management: A lifecycle based requirements analysis. In *Proceedings of the Workshop on Semantic Business Process and Product Lifecycle Management (SBPM-2007)*, volume 251, pages 1613–0073. Citeseer, 2007.

Trademarks

IBM, the IBM logo, ibm.com, and WebSphere are trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at www.ibm.com/legal/copytrade.shtml.